

Cheatsheet v2

Code-grounded summary of what is implemented, how the pipeline works, and how to extend it in a code test.

1. What Is Implemented

Area	Implemented now
Types	Integer, Float, Boolean, String, internal Void.
Variables	First assignment fixes the variable type; later assignments must match.
Arithmetic	Integer: <code>++*/</code> . Float: <code>+.*./</code> . Integer division is floor division.
Comparison	Only <code>==</code> and <code>!=</code> . Both operands must be numeric and same type. Result is Boolean.
Unary minus	<code>-x</code> and <code>-.x</code> . Parser desugars them to <code>0-x</code> and <code>0.0-.x</code> .
Control flow	<code>if/else</code> , <code>while</code> . Condition must be Boolean.
Functions	<code>defname(params)\{...\}</code> ; call-by-value; <code>returnexpr;;</code> no recursion support.
Built-in	<code>print(expr);</code> prints <code>value:Type</code> .
Outputs	Tokens, parse tree, AST, type table, execution output.
Errors	Lexer, parser, type checker, interpreter all report line and column.
Tests	77 regression tests in <code>tests/test_pipeline.py</code> .

2. Pipeline You Must Remember

1. `components/lexica.py`: source text to token list.
2. `components/parser.py`: token list to AST rooted at `Program`.
3. `components/parse_tree_printer.py`: AST to grammar-style parse tree text.
4. `components/ast_printer.py`: AST to readable AST text.
5. `components/type_checker.py`: AST to typed global symbol table.
6. `components/interpreter.py`: AST to runtime output.
7. `components/pipeline.py`: glue used by CLI, UI, and tests.

Core rule for code tests: if you add a language feature, you usually touch four places:

1. token model in `components/tokens.py`
2. lexer in `components/lexica.py`
3. parser and possibly AST/printers in `components/parser.py`, `components/ast_nodes.py`, `components/parse_tree_printer.py`, `components/ast_printer.py`
4. semantics in `components/type_checker.py` and `components/interpreter.py`

3. Exact File Map

File	Responsibility
<code>components/tokens.py</code>	<code>TokenType</code> enum and <code>Token</code> dataclass.
<code>components/lexica.py</code>	Character scanning, keyword map, operator tokenization, line/column tracking.
<code>components/ast_nodes.py</code>	Dataclasses for <code>Program</code> , <code>Assign</code> , <code>If</code> , <code>While</code> , <code>FunctionDef</code> , <code>FunctionCall</code> , <code>Return</code> , <code>BinaryOp</code> , <code>Literal</code> , <code>Identifier</code> , <code>Block</code> .
<code>components/parser.py</code>	Recursive-descent grammar and unary-minus desugaring.
<code>components/parse_tree_printer.py</code>	Grammar-oriented concrete tree rendering.
<code>components/ast_printer.py</code>	Human-readable AST rendering.
<code>components/symbol_table.py</code>	<code>LanguageType</code> , scoped symbol table, function and variable symbol records.
<code>components/type_checker.py</code>	Type inference, function registration, arg checking, return-path checking, no recursion rule.
<code>components/interpreter.py</code>	Runtime environments, statement execution, expression evaluation, built-in <code>print</code> .
<code>components/pipeline.py</code>	End-to-end run and stage formatting.

main.py	CLI entry.
ui.py	Tkinter UI showing all five outputs.
tests/test_pipeline.py	Unit and integration tests.

4. Semantics That Examiners Can Ask About

- Variable type is inferred on first assignment in `components/type_checker.py`, method `_assign_variable_type`.
- Function parameter types are inferred on first call. If a function is never called, `_infer_param_types_from_body` scans operators in the body and defaults unresolved params to Integer.
- Recursive calls are rejected in `_infer_function_call_type` via `_active_calls`.
- Non-void functions must return on all paths. Checked by `_definitely_returns`.
- `==` and `!=` are only for arithmetic operands, not String or Boolean.
- Block scopes are isolated: new names inside a child scope are not visible outside; reassigning an existing outer name updates the outer one.
- `print` returns Void, so it cannot be assigned or passed where a value is required.
- Interpreter trusts the type checker for most type safety and mainly handles runtime behavior such as division by zero and environment lookup.

5. Current Grammar Snapshot

```

program -> statement* EOF
statement -> func_def
           | if_stmt
           | while_stmt
           | return_stmt
           | assignment
           | expr ";"

assignment -> IDENTIFIER "=" expr ";"
if_stmt -> "if" "(" expr ")" block ("else" block)?
while_stmt -> "while" "(" expr ")" block
func_def -> "def" IDENTIFIER "(" params? ")" block
return_stmt -> "return" expr ";"
block -> "{" statement* "}"
params -> IDENTIFIER "(" IDENTIFIER)*
args -> expr "(" IDENTIFIER)*
expr -> additive ("==" | "!=") additive)?
additive -> term ("+" | "-" | "+" | "-") term)*
term -> factor ("*" | "/" | "*" | "/") factor)*
factor -> "-" factor
        | "-" factor
        | literal
        | IDENTIFIER "(" args? ")"
        | "(" expr ")"

```

6. Fast Extension Pattern

When the exam asks “add a new operator / new statement / new literal / new built-in”, use this order:

1. Add token kinds in `components/tokens.py`.
2. Teach the lexer how to scan the new lexeme in `components/lexer.py`.
3. Extend the parser grammar in `components/parser.py`.
4. If the feature needs a new AST node, add it in `components/ast_nodes.py`.
5. Update both printers so Stage 2 and Stage 3 still work.
6. Add type rules in `components/type_checker.py`.
7. Add runtime behavior in `components/interpreter.py`.
8. Add tests in `tests/test_pipeline.py`.

7. Exam Scenario Playbook

Legend: **ADD** means insert new code. **MODIFY** means edit existing code. **NO CHANGE** means explicitly leave that file alone.

Likely Code-Test Questions

1. Add a new arithmetic operator such as `%`.
2. Add a new comparison operator such as `<=`.
3. Add Boolean operators such as `and/or`.
4. Add comment syntax.
5. Relax equality so Strings and Booleans can also be compared.
6. Add a new built-in function such as `len()`.
7. Add a new statement form such as `for`.
8. Add explicit type annotations.
9. Add unary logical `not` with correct precedence.
10. Add relational operators but keep old equality behavior unchanged.
11. Add a `do-while` loop or post-test loop.
12. Add block comments without breaking line counting.
13. Add string concatenation but do not allow String with Integer accidentally.
14. Add function overloading and explain why the current symbol table design blocks it.
15. Allow recursion and explain why the current type checker rejects it.
16. Add a new output tab or pipeline display without changing the language semantics.

Final content audit:

- High-probability scenarios state the exact file.
- Each scenario marks **ADD**, **MODIFY**, or **NO CHANGE**.
- Each scenario gives the current target line region.
- Each major scenario gives a concrete patch-style snippet.
- Grammar-sensitive cases are marked as design-sensitive, not fake drop-in patches.

Harder Examiner Traps

- `//` comments: mostly lexer work, but insert carefully because `/` and `/.` already exist in `components/lexica.py` lines 83–99.
- `and/or`: not only lexer work; precedence changes force parser refactoring in `components/parser.py` lines 169–199.
- `for`: not only parser work; also AST, printers, type checker, interpreter, and tests.
- recursion: semantic redesign, not a quick patch; current blocker is `components/type_checker.py` lines 154–191 plus `_active_calls` at lines 41–45.
- string equality: mostly type-checker work; interpreter equality already works.
- `len()`: built-in function case; parser syntax already exists.

Case A: Add a New Binary Operator such as `%`

Typical question: “Add modulo for integers only.”

1. In `components/tokens.py`, add `PERCENT` to `TokenType`.
2. In `components/lexica.py`, add `"%":TokenType.PERCENT` to `SINGLE_CHAR_TOKENS`.
3. In `components/parser.py`, treat `%` like `*` and `/` inside `_term()` because it has multiplicative precedence.
4. No new AST node is needed because `BinaryOp` already stores `op`, `left`, `right`.
5. In `components/parse_tree_printer.py`, add `%` to the operator set handled by `_term_node()`.
6. In `components/type_checker.py`, update `_infer_binary_type()` so `%` requires two Integers and returns Integer.
7. In `components/interpreter.py`, update `_evaluate_binary()` with modulo runtime logic and zero check.
8. In `tests/test_pipeline.py`, add:

`components/tokens.py`

ADD Current target lines: 24–35

```

class TokenType(Enum):
    ~ # Operators
    ~ PLUS = auto()
    ~ MINUS = auto()
    ~ STAR = auto()
    ~ SLASH = auto()
    + PERCENT = auto()
      PLUS_DOT = auto()

```

components/lexica.py

MODIFY Current target lines: 27–35

```

SINGLE_CHAR_TOKENS: dict[str, TokenType] = {
    "=": TokenType.ASSIGN,
    "(": TokenType.LPAREN,
    ")": TokenType.RPAREN,
    "{": TokenType.LBRACE,
    "}": TokenType.RBRACE,
    ",": TokenType.COMMA,
    ";": TokenType.SEMICOLON,
    + "%": TokenType.PERCENT,
}

```

components/parser.py

MODIFY Current target lines: 190–199

```

def _term(self) -> ASTNode:
    node = self._factor()
    while (
        self._check(TokenType.STAR)
        or self._check(TokenType.SLASH)
    + or self._check(TokenType.PERCENT)
        or self._check(TokenType.STAR_DOT)
        or self._check(TokenType.SLASH_DOT)
    ):
        op_tok = self._advance()
        right = self._factor()
        node = BinaryOp(op=op_tok.lexeme, left=node, right=right,
                        line=op_tok.line, column=op_tok.column)
    return node

```

components/parse_tree_printer.py

MODIFY Current target lines: 185–195

```

def _term_node(self, node: ASTNode) -> ParseTreeNode:
    ~ if isinstance(node, BinaryOp) and node.op in {"*", "/", "%", ".*", "/."}:
        return ParseTreeNode(
            "term",
            [
                self._term_node(node.left),
                ParseTreeNode(f"'{node.op}'"),
                self._factor_node(node.right),
            ],
        )
    return ParseTreeNode("term", [self._factor_node(node)])

```

components/type_checker.py

MODIFY Current target lines: 341–355

```

def _infer_binary_type(self, node: BinaryOp, left_type: LanguageType, right_type: LanguageType) -> LanguageType:
    ~ if node.op in {"+", "-", "*", "%"}:
        if left_type != LanguageType.INTEGER or right_type != LanguageType.INTEGER:
            raise TypeCheckError(
                self._error_at(node, f"Operator '{node.op}' requires two Integer operands")
            )
        return LanguageType.INTEGER

```

components/interpreter.py

MODIFY Current target lines: 182–207

```

def _evaluate_binary(self, node: BinaryOp, left_value: object, right_value: object) -> object:
    if node.op == "/":
        if int(right_value) == 0:
            raise InterpreterError(self._error_at(node, "Division by zero"))
        return int(left_value) // int(right_value)
    + if node.op == "%":
    + if int(right_value) == 0:
    + raise InterpreterError(self._error_at(node, "Division by zero"))
    + return int(left_value) % int(right_value)

```

Modulo visual before/after

Before	After
<pre> if node.op == "/": if int(right_value) == 0: raise InterpreterError(self._error_at(node, "Division by zero")) return int(left_value) // int(right_value) </pre>	<pre> if node.op == "/": if int(right_value) == 0: raise InterpreterError(self._error_at(node, "Division by zero")) return int(left_value) // int(right_value) if node.op == "%": if int(right_value) == 0: raise InterpreterError(self._error_at(node, "Division by zero")) return int(left_value) % int(right_value) </pre>

tests/test_pipeline.py

ADD Current target lines: insert before line 453

```

def test_modulo_integer(self) -> None:
    result = run_pipeline("x = 7 % 3; print(x);")
    self.assertEqual(result.outputs, ["1 : Integer"])

def test_modulo_by_zero_raises_error(self) -> None:
    with self.assertRaises(InterpreterError):
        run_pipeline("x = 7 % 0;")

```

Case B: Add a New Comparison such as <= or >=

Typical question: "Add <= for numeric same-type operands."

1. components/tokens.py: add LESS_EQUAL or GREATER_EQUAL.
2. components/lexica.py: scan the two-character operator.
3. components/parser.py: extend _expr() so it accepts the new comparison token.
4. components/parse_tree_printer.py: include the operator in _expr_node().
5. components/type_checker.py: same numeric same-type rule as == and !=, return Boolean.
6. components/interpreter.py: implement Python runtime comparison.
7. tests/test_pipeline.py: one success test and one type error test.

components/tokens.py

ADD Current target lines: 33–35

```

class TokenType(Enum):
    STAR_DOT = auto()
    SLASH_DOT = auto()
    EQUAL_EQUAL = auto()
    BANG_EQUAL = auto()
+ LESS_EQUAL = auto()
    ASSIGN = auto()

```

components/lexica.py

MODIFY Current target lines: 73–82

```

+ if char == "<":
+ if self._match("="):
+ return self._make_token(TokenType.LESS_EQUAL)
+ raise LexerError(self._error_message("Unexpected '<'. Did you mean '<=?'"))

```

Comparison visual before/after

Before	After
<pre> if char == "=": if self._match("="): return token(EQUAL_EQUAL) </pre>	<pre> if char == "=": if self._match("="): return token(EQUAL_EQUAL) if char == "<": if self._match("="): return token(LESS_EQUAL) raise LexerError(self._error_message("Unexpected '<'. Did you mean '<=?'")) </pre>

```
def _expr(self) -> ASTNode:
    node = self._additive()
    if (
        self._check(TokenType.EQUAL_EQUAL)
        or self._check(TokenType.BANG_EQUAL)
        + or self._check(TokenType.LESS_EQUAL)
    ):
        op_tok = self._advance()
        right = self._additive()
        node = BinaryOp(op=op_tok.lexeme, left=node, right=right,
                        line=op_tok.line, column=op_tok.column)
    return node
```

```
def _expr_node(self, node: ASTNode) -> ParseTreeNode:
    ~ if isinstance(node, BinaryOp) and node.op in {"==", "!=", "<="}:
        child = ParseTreeNode(
            "comparison",
            [
                self._additive_node(node.left),
                ParseTreeNode(f'"{node.op}"'),
                self._additive_node(node.right),
            ],
        )
        return ParseTreeNode("expr", [child])
    return ParseTreeNode("expr", [self._additive_node(node)])
```

```
if node.op in {"==", "!=", "<="}:
    if not self._is_numeric(left_type) or not self._is_numeric(right_type):
        raise TypeCheckError(
            self._error_at(node, f"Operator '{node.op}' requires two arithmetic operands")
        )
    if left_type != right_type:
        raise TypeCheckError(
            self._error_at(node, f"Operator '{node.op}' requires operands of the same type, got {left_type.value} and {right_type.value}")
        )
    return LanguageType.BOOLEAN
```

```
+ if node.op == "<=":
+ return left_value <= right_value
```

```
def test_less_equal_true(self) -> None:
    result = run_pipeline("print(2 <= 3);")
    self.assertEqual(result.outputs, ["true : Boolean"])

def test_less_equal_mixed_types_raises_type_error(self) -> None:
    with self.assertRaises(TypeError):
        run_pipeline("print(2 <= 3.0);")
```

Case C: Add Boolean Operators such as and / or

Important: this is not just lexer work; it changes precedence and type rules.

1. Add keyword tokens in components/tokens.py.
2. Add them to KEYWORDS in components/lexica.py.
3. In components/parser.py, add new precedence functions, for example:

```
expr -> or_expr
or_expr -> and_expr ("or" and_expr)*
and_expr -> equality ("and" equality)*
```

1. Rewire _expr() to call the new highest-level function.
2. Update parse tree printer to render the new grammar layers.
3. In components/type_checker.py, require Boolean operands and return Boolean.
4. In components/interpreter.py, implement logical behavior.
5. Add tests for precedence such as trueorfalseandfalse.

```

class TokenType(Enum):
    IF = auto()
    ELSE = auto()
    WHILE = auto()
    DEF = auto()
    RETURN = auto()
+ AND = auto()
+ OR = auto()

```

components/lexica.py

MODIFY Current target lines: 8–16

```

KEYWORDS: dict[str, TokenType] = {
    "if": TokenType.IF,
    "else": TokenType.ELSE,
    "while": TokenType.WHILE,
    "def": TokenType.DEF,
    "return": TokenType.RETURN,
+ "and": TokenType.AND,
+ "or": TokenType.OR,
    "true": TokenType.BOOL_LITERAL,
    "false": TokenType.BOOL_LITERAL,
}

```

components/parser.py

MODIFY Current target lines: 169–199

```

def _expr(self) -> ASTNode:
~ return self._or_expr()

def _or_expr(self) -> ASTNode:
    node = self._and_expr()
    while self._check(TokenType.OR):
        op_tok = self._advance()
        right = self._and_expr()
        node = BinaryOp(op=op_tok.lexeme, left=node, right=right,
                        line=op_tok.line, column=op_tok.column)
    return node

def _and_expr(self) -> ASTNode:
    node = self._equality_expr()
    while self._check(TokenType.AND):
        op_tok = self._advance()
        right = self._equality_expr()
        node = BinaryOp(op=op_tok.lexeme, left=node, right=right,
                        line=op_tok.line, column=op_tok.column)
    return node

def _equality_expr(self) -> ASTNode:
    node = self._additive()
    if self._check(TokenType.EQUAL_EQUAL) or self._check(TokenType.BANG_EQUAL):
        op_tok = self._advance()
        right = self._additive()
        node = BinaryOp(op=op_tok.lexeme, left=node, right=right,
                        line=op_tok.line, column=op_tok.column)
    return node

```

components/parse_tree_printer.py

MODIFY

```

def _expr_node(self, node: ASTNode) -> ParseTreeNode:
    return ParseTreeNode("expr", [self._or_node(node)])

def _or_node(self, node: ASTNode) -> ParseTreeNode:
    if isinstance(node, BinaryOp) and node.op == "or":
        return ParseTreeNode("or_expr", [self._or_node(node.left), ParseTreeNode("'or'"), self._and_node(node.right)])
    return ParseTreeNode("or_expr", [self._and_node(node)])

def _and_node(self, node: ASTNode) -> ParseTreeNode:
    if isinstance(node, BinaryOp) and node.op == "and":
        return ParseTreeNode("and_expr", [self._and_node(node.left), ParseTreeNode("'and'"), self._equality_node(node.right)])
    return ParseTreeNode("and_expr", [self._equality_node(node)])

def _equality_node(self, node: ASTNode) -> ParseTreeNode:
    if isinstance(node, BinaryOp) and node.op in {"==", "!="}:
        return ParseTreeNode("equality", [self._additive_node(node.left), ParseTreeNode(f"'{node.op}'"), self._additive_node(node.right)])
    return ParseTreeNode("equality", [self._additive_node(node)])

```

components/ast_printer.py

MODIFY Current target lines: 99–104

```

# no new visitor is needed if 'and'/'or' still use BinaryOp
# existing _visit_BinaryOp already prints the operator string

```

components/type_checker.py

MODIFY Current target lines: 335–382

```

+ if node.op in {"and", "or"}:
+ if left_type != LanguageType.BOOLEAN or right_type != LanguageType.BOOLEAN:
+ raise TypeCheckError(
+ self._error_at(node, f"Operator '{node.op}' requires two Boolean operands")
+ )
+ return LanguageType.BOOLEAN

```

components/interpreter.py

MODIFY Current target lines: 182–207

```
+ if node.op == "and":
+ return bool(left_value) and bool(right_value)
+ if node.op == "or":
+ return bool(left_value) or bool(right_value)
```

tests/test_pipeline.py

ADD Current target lines: insert before line 453

```
def test_boolean_and_or_precedence(self) -> None:
    result = run_pipeline("print(true or false and false);")
    self.assertEqual(result.outputs, ["true : Boolean"])

def test_boolean_and_rejects_integer(self) -> None:
    with self.assertRaises(TypeError):
        run_pipeline("print(true and 1);")
```

Case D: Add a New Statement such as for

This usually needs a new AST node.

1. In components/tokens.py, add FOR.
2. In components/lexica.py, add "for" to KEYWORDS.
3. In components/ast_nodes.py, add a For dataclass with fields such as init, condition, update, body.
4. In components/parser.py, extend _statement() to dispatch to _for_stmt().
5. Parse the exact grammar the exam gives.
6. In components/parse_tree_printer.py and components/ast_printer.py, add visitors for For.
7. In components/type_checker.py, add a new branch in _check_statement():

components/tokens.py

ADD Current target lines: 17–22

```
class TokenType(Enum):
    IF = auto()
    ELSE = auto()
    WHILE = auto()
    DEF = auto()
+ FOR = auto()
    RETURN = auto()
```

components/lexica.py

MODIFY Current target lines: 8–16

```
KEYWORDS: dict[str, TokenType] = {
    "if": TokenType.IF,
    "else": TokenType.ELSE,
    "while": TokenType.WHILE,
    "def": TokenType.DEF,
    "return": TokenType.RETURN,
+ "for": TokenType.FOR,
    "true": TokenType.BOOL_LITERAL,
    "false": TokenType.BOOL_LITERAL,
}
```

components/ast_nodes.py

ADD Current target lines: 127–136

```
@dataclass
class For(ASTNode):
    init: ASTNode
    condition: ASTNode
    update: ASTNode
    body: Block
```

components/parser.py

MODIFY Current target lines: 62–88

```
def _statement(self) -> ASTNode:
    tok = self._peek()
+ if tok.token_type == TokenType.FOR:
+ return self._for_stmt()
    if tok.token_type == TokenType.DEF:
        return self._func_def()
    if tok.token_type == TokenType.IF:
        return self._if_stmt()
```

components/parser.py

ADD Current target lines: after 139

```
def _assignment_no_semicolon(self) -> Assign:
    name_tok = self._advance()
    self._advance() # '='
    value = self._expr()
```

```

    return Assign(name=name_tok.lexeme, value=value, line=name_tok.line, column=name_tok.column)

def _for_stmt(self) -> For:
    tok = self._advance() # 'for'
    self._expect(TokenType.LPAREN, "Expected '(' after 'for'")
    init = self._assignment_no_semicolon()
    self._expect(TokenType.SEMICOLON, "Expected ';' after for initializer")
    condition = self._expr()
    self._expect(TokenType.SEMICOLON, "Expected ';' after for condition")
    update = self._assignment_no_semicolon()
    self._expect(TokenType.RPAREN, "Expected ')' after for clauses")
    body = self._block()
    return For(init=init, condition=condition, update=update, body=body, line=tok.line, column=tok.column)

```

components/parse_tree_printer.py

MODIFY Current target lines: 64–83

```

+ if isinstance(node, For):
+ return ParseTreeNode("statement", [self._for_node(node)])
+
+ def _assignment_inline_node(self, node: Assign) -> ParseTreeNode:
+ return ParseTreeNode("assignment_inline", [ParseTreeNode(f"IDENTIFIER({node.name})", ParseTreeNode("'='"), self._expr_node(node.value))])
+
+ def _for_node(self, node: For) -> ParseTreeNode:
+ return ParseTreeNode(
+ "for_stmt",
+ [
+ ParseTreeNode("'for'"),
+ ParseTreeNode("("),
+ self._assignment_inline_node(node.init),
+ ParseTreeNode(";"),
+ self._expr_node(node.condition),
+ ParseTreeNode(";"),
+ self._assignment_inline_node(node.update),
+ ParseTreeNode(")"),
+ self._block_node(node.body),
+ ],
+ )

```

components/ast_printer.py

MODIFY Current target lines: 77–82

```

+ def _visit_For(self, node: For, lines: list[str], level: int) -> None:
+ lines.append(f"{self._pad(level)}For")
+ lines.append(f"{self._pad(level + 1)}init:")
+ self._visit(node.init, lines, level + 2)
+ lines.append(f"{self._pad(level + 1)}condition:")
+ self._visit(node.condition, lines, level + 2)
+ lines.append(f"{self._pad(level + 1)}update:")
+ self._visit(node.update, lines, level + 2)
+ lines.append(f"{self._pad(level + 1)}body:")
+ self._visit(node.body, lines, level + 2)

```

components/type_checker.py

MODIFY Current target lines: 70–120

```

+ if isinstance(node, For):
+ loop_scope = scope.child_scope()
+ self._check_statement(node.init, loop_scope, function_context)
+ condition_type = self._infer_expr_type(node.condition, loop_scope)
+ if condition_type != LanguageType.BOOLEAN:
+ raise TypeCheckError(self._error_at(node.condition, "For condition must have type Boolean"))
+ self._check_statement(node.update, loop_scope, function_context)
+ self._check_block(node.body, loop_scope.child_scope(), function_context)
+ return

```

components/interpreter.py

MODIFY Current target lines: 87–123

```

+ if isinstance(node, For):
+ loop_env = _Environment(parent=environment)
+ self._execute_statement(node.init, loop_env)
+ while True:
+ condition = self._evaluate(node.condition, loop_env)
+ if not isinstance(condition, bool):
+ raise InterpreterError(self._error_at(node.condition, "For condition must evaluate to Boolean"))
+ if not condition:
+ break
+ self._execute_block(node.body, _Environment(parent=loop_env))
+ self._execute_statement(node.update, loop_env)
+ return

```

tests/test_pipeline.py

ADD Current target lines: insert before line 453

```

def test_for_loop_counts_down(self) -> None:
    result = run_pipeline("for (i = 3; i != 0; i = i - 1) { print(i); }")
    self.assertEqual(result.outputs, ["3 : Integer", "2 : Integer", "1 : Integer"])

```

Case E: Add a New Literal Type

Example question: “Add character literals” or “add nil”.

1. components/tokens.py: new literal token.
2. components/lexica.py: new scanner branch.
3. components/parser.py: new CharLiteral(value=tok.lexeme[1:-1], line=tok.line, column=tok.column) construction in _factor().
4. components/symbol_table.py: add a new LanguageType entry if needed.
5. components/type_checker.py: update _literal_type().
6. components/interpreter.py: update _runtime_type() and maybe _format_value().
7. Add tests for lexing, typing, and printing.

components/tokens.py

ADD Current target lines: 8–12

```
class TokenType(Enum):
    INT_LITERAL = auto()
    FLOAT_LITERAL = auto()
    BOOL_LITERAL = auto()
    STRING_LITERAL = auto()
+ CHAR_LITERAL = auto()
```

components/lexica.py

MODIFY Current target lines: 101–110

```
+ if char == "'":
+ return self._char_literal()

def _char_literal(self) -> Token:
    if self._is_at_end() or self._peek() == "\n":
        raise LexerError(self._error_message("Unterminated char literal"))
    self._advance() # consume char
    if self._peek() != "'":
        raise LexerError(self._error_message("Char literal must contain exactly one character"))
    self._advance() # closing quote
    return self._make_token(TokenType.CHAR_LITERAL)
```

components/ast_nodes.py

ADD Current target lines: 22–30

```
@dataclass
class CharLiteral(ASTNode):
    value: str
```

components/parser.py

MODIFY Current target lines: 226–244

```
+ if tok.token_type == TokenType.CHAR_LITERAL:
+ self._advance()
+ return CharLiteral(value=tok.lexeme[1:-1], line=tok.line, column=tok.column)
```

components/type_checker.py

MODIFY Current target lines: 131–152 and 398–407

```
if isinstance(node, CharLiteral):
    return LanguageType.CHAR
```

components/symbol_table.py

ADD Current target lines: 7–12

```
class LanguageType(Enum):
    INTEGER = "Integer"
    FLOAT = "Float"
    BOOLEAN = "Boolean"
    STRING = "String"
+ CHAR = "Char"
    VOID = "Void"
```

components/interpreter.py

MODIFY Current target lines: 129–144 and 217–227

```
if isinstance(node, CharLiteral):
    return node.value

def _runtime_type(value: object) -> LanguageType:
    if isinstance(value, str):
        # with only this patch, runtime strings and chars share Python 'str'
        return LanguageType.STRING
```

tests/test_pipeline.py

ADD Current target lines: insert before line 453

```
def test_char_literal_prints(self) -> None:
    result = run_pipeline("print('a');")
    self.assertEqual(result.outputs, ['a' : String'])
```

Case F: Add Comments

This is lexer-only unless the specification says comments appear in output.

1. In components/lexica.py, inside `_scan_token()`, detect the comment opener.
2. Consume characters until newline or comment terminator.
3. Return `None` so the comment does not emit tokens.
4. Add lexer tests proving comments are skipped and line counting stays correct.

components/lexica.py

MODIFY Current target lines: 83–99

```
+ if char == "/" and self._peek() == "/":
+ self._advance() # consume second slash
+ while self._peek() != "\n" and not self._is_at_end():
+ self._advance()
+ return None
```

tests/test_pipeline.py

ADD Current target lines: insert before line 85

```
def test_line_comment_is_skipped(self) -> None:
    result = run_pipeline("x = 1; // ignore this\nprint(x);")
    self.assertEqual(result.outputs, ["1 : Integer"])
```

Case G: Allow String or Boolean Equality

Current behavior: `==` and `!=` only accept arithmetic operands.

1. In components/type_checker.py, edit `_infer_binary_type()`.
2. Replace the current “numeric only” rule for `==/!=` with:

```
if left_type != right_type:
    error
if left_type == LanguageType.VOID:
    error
return LanguageType.BOOLEAN
```

1. Interpreter changes are probably unnecessary because Python already compares strings and booleans.
2. Update or replace old failing tests: `test_boolean_equality_raises_type_error`, `test_string_equality_raises_type_error`.
3. Add success tests for `"a"=="a"` and `true!=false`.

components/type_checker.py

MODIFY Current target lines: 365–380

```
if node.op in {"==", "!="}:
    if left_type != right_type:
        raise TypeCheckError(
            self._error_at(
                node,
                f"Operator '{node.op}' requires operands of the same type, got {left_type.value} and {right_type.value}",
            )
        )
    if left_type == LanguageType.VOID:
        raise TypeCheckError(
            self._error_at(node, f"Operator '{node.op}' cannot compare Void values")
        )
    return LanguageType.BOOLEAN
```

Equality rule visual before/after

Before	After
<pre> if node.op in {"==", "!="}: if not numeric(left_type, right_type): raise TypeCheckError(self._error_at(node, f"Operator '{node.op}' requires numeric operands")) if left_type != right_type: raise TypeCheckError(self._error_at(node, f"Operator '{node.op}' requires operands of the same type")) return LanguageType.BOOLEAN </pre>	<pre> if node.op in {"==", "!="}: if left_type != right_type: raise TypeCheckError(self._error_at(node, f"Operator '{node.op}' requires operands of the same type, got {left_type.value} and {right_type.value}") if left_type == LanguageType.VOID: raise TypeCheckError(self._error_at(node, f"Operator '{node.op}' cannot compare Void values")) return LanguageType.BOOLEAN </pre>

components/interpreter.py

NO CHANGE Current target lines: 203–206

```

# No runtime change required.
# Existing Python equality on values is already sufficient.

```

tests/test_pipeline.py

MODIFY Current target lines: 241–245 and 423–426

```

def test_boolean_equality_allowed(self) -> None:
    result = run_pipeline("print(true != false);")
    self.assertEqual(result.outputs, ["true : Boolean"])

def test_string_equality_allowed(self) -> None:
    result = run_pipeline('print("a" == "a");')
    self.assertEqual(result.outputs, ["true : Boolean"])

```

Case H: Add a New Built-in Function

Example question: “Add input()” or “add len(s)”.

1. If the syntax is still a normal call, no parser change is needed.
2. In components/type_checker.py, extend `_infer_function_call_type()` with a new built-in branch, like the current print.
3. Validate arity and operand types there.
4. In components/interpreter.py, extend `_call_function()` with a runtime built-in branch.
5. Add tests for valid calls and invalid arguments.

components/type_checker.py

MODIFY Current target lines: 154–206

```

def _infer_function_call_type(self, node: FunctionCall, scope: SymbolTable) -> LanguageType:
    if node.name == self._BUILTIN_PRINT:
        return self._infer_builtin_print_type(node, scope)
    if node.name == "len":
        return self._infer_builtin_len_type(node, scope)
    function_state = self._functions.get(node.name)
    if function_state is None:
        raise TypeCheckError(self._error_at(node, f"Undefined function: {node.name}"))

def _infer_builtin_len_type(self, node: FunctionCall, scope: SymbolTable) -> LanguageType:
    if len(node.args) != 1:
        raise TypeCheckError(self._error_at(node, f"Built-in function 'len' expects 1 argument, got {len(node.args)}"))
    arg_type = self._infer_expr_type(node.args[0], scope)
    if arg_type != LanguageType.STRING:
        raise TypeCheckError(self._error_at(node.args[0], "Argument to 'len' must be String"))
    return LanguageType.INTEGER

```

components/interpreter.py

MODIFY Current target lines: 146–180

```

def _call_function(self, node: FunctionCall, environment: _Environment) -> object:
    if node.name == self._BUILTIN_PRINT:
        return self._call_builtin_print(node, environment)
    if node.name == "len":
        return self._call_builtin_len(node, environment)
    function = self._functions.get(node.name)
    if function is None:
        raise InterpreterError(self._error_at(node, f"Undefined function: {node.name}"))

```

```
def _call_builtin_len(self, node: FunctionCall, environment: _Environment) -> int:
    if len(node.args) != 1:
        raise InterpreterError(self._error_at(node, f"Built-in function 'len' expects 1 argument, got {len(node.args)}"))
    value = self._evaluate(node.args[0], environment)
    return len(str(value))
```

tests/test_pipeline.py

ADD Current target lines: insert before line 453

```
def test_len_builtin_returns_integer(self) -> None:
    result = run_pipeline('print(len("plc"));')
    self.assertEqual(result.outputs, ["3 : Integer"])
```

Case I: Add Recursion

Current blocker: components/type_checker.py rejects recursive calls.

1. Remove or redesign the `_active_calls` rejection in `_infer_function_call_type()`.
2. Keep placeholder function symbols registered before body checking, which the code already does.
3. Make sure parameter types and return type can stabilize before recursive bodies are fully checked. This is the hard part.
4. Minimum safe exam answer: state that lexer/parser/interpreter already support calling functions by name, but the type checker must change from single-pass body checking to a fixpoint or staged inference strategy.
5. Treat the test below as the *target regression test after redesign*, not as a drop-in patch that the current checker can already pass.

components/type_checker.py

MODIFY Current target lines: 154–163

```
# current rejection point in the real checker:
if node.name in self._active_calls:
    raise TypeCheckError(self._error_at(node, f"Recursive function calls are not supported: {node.name}"))
```

components/type_checker.py

MODIFY Current target lines: 188–191 and 208–246

```
+ Accuracy note for this repository:
+ recursion is not a bounded patch.
+ A correct implementation requires redesigning function-body checking so
+ re-entrant calls do not recurse forever while return/parameter types are
+ still unresolved.
+ In an exam, state this explicitly instead of pretending the current
+ single-pass checker can be made recursive with a 2-line edit.
```

tests/test_pipeline.py

ADD Current target lines: insert before line 453

```
def test_recursive_factorial(self) -> None:
    result = run_pipeline(
        "def fact(n) { if (n == 0) { return 1; } else { return n * fact(n - 1); } } print(fact(4));"
    )
    self.assertEqual(result.outputs, ["24 : Integer"])
```

Case J: Add Explicit Type Annotations

Example question: “Support `x:Integer=5;`”

1. Add tokens for `:` and possibly type keywords.
2. Lexer: add the punctuation and keywords.
3. Parser: extend assignment grammar.
4. AST: add a field to store the declared type on assignments.
5. Symbol table: reuse existing `LanguageType`.
6. Type checker: compare declared type against inferred expression type.
7. Interpreter: likely unchanged.
8. Add tests for both valid typed assignments and mismatches.

components/tokens.py

ADD Current target lines: 37–43

```
class TokenType(Enum):
    LPAREN = auto()
    RPAREN = auto()
    LBRACE = auto()
    RBRACE = auto()
    COMMA = auto()
```

```
+ COLON = auto()
+ TYPE_INTEGER = auto()
+ TYPE_FLOAT = auto()
```

components/lexica.py

MODIFY Current target lines: 27–35 and 8–16

```
SINGLE_CHAR_TOKENS = {
    "=": TokenType.ASSIGN,
    "(": TokenType.LPAREN,
    ")": TokenType.RPAREN,
    "{": TokenType.LBRACE,
    "}": TokenType.RBRACE,
    ",": TokenType.COMMA,
+   "(": TokenType.COLON,
+   ";": TokenType.SEMICOLON,
}

KEYWORDS = {
    "if": TokenType.IF,
    "else": TokenType.ELSE,
    "while": TokenType.WHILE,
    "def": TokenType.DEF,
    "return": TokenType.RETURN,
+   "Integer": TokenType.TYPE_INTEGER,
+   "Float": TokenType.TYPE_FLOAT,
    "true": TokenType.BOOL_LITERAL,
    "false": TokenType.BOOL_LITERAL,
}
```

components/type_checker.py

ADD Current target lines: after 410

```
def _language_type_from_name(self, name: str) -> LanguageType:
    mapping = {
        "Integer": LanguageType.INTEGER,
        "Float": LanguageType.FLOAT,
        "Boolean": LanguageType.BOOLEAN,
        "String": LanguageType.STRING,
    }
    try:
        return mapping[name]
    except KeyError as error:
        raise TypeCheckError(f"Unknown declared type: {name}") from error
```

components/ast_nodes.py

MODIFY Current target lines: 66–74

```
class Assign(ASTNode):
    name: str
+   declared_type: str | None = None
    value: ASTNode
```

components/parser.py

MODIFY Current target lines: 90–97

```
# support: x : Integer = 5;
name_tok = self._advance()
declared_type = None
if self._check(TokenType.COLON):
    self._advance()
    declared_type = self._advance().lexeme
self._advance() # '='
value = self._expr()
return Assign(
    name=name_tok.lexeme,
    declared_type=declared_type,
    value=value,
    line=name_tok.line,
    column=name_tok.column,
)
```

components/type_checker.py

MODIFY Current target lines: 315–333

```
if node.declared_type is not None:
    declared = self._language_type_from_name(node.declared_type)
    if declared != value_type:
        raise TypeCheckError(self._error_at(node, f"Declared type {declared.value} does not match {value_type.value}"))
```

tests/test_pipeline.py

ADD Current target lines: insert before line 453

```
def test_typed_assignment_matches(self) -> None:
    result = run_pipeline("x: Integer = 5; print(x);")
    self.assertEqual(result.outputs, ["5 : Integer"])

def test_typed_assignment_mismatch_raises(self) -> None:
    with self.assertRaises(TypeCheckError):
        run_pipeline('x: Integer = "hello";')
```

Case K: Add Unary not

Trap: this is not a binary operator. Put it near `_factor()` or add a separate boolean-unary precedence level.

1. `components/tokens.py`: add NOT.
2. `components/lexica.py`: add "not" to KEYWORDS.
3. `components/parser.py`: parse unary `notexpr`.
4. `components/ast_nodes.py`: either reuse `FunctionCall`-style workaround or better add a dedicated unary node.
5. `components/type_checker.py`: require Boolean operand, return Boolean.
6. `components/interpreter.py`: return Python `not`.
7. `tests/test_pipeline.py`: add precedence and type-error tests.

`components/tokens.py`

ADD Current target lines: 17–22

```
class TokenType(Enum):
    IF = auto()
    ELSE = auto()
    WHILE = auto()
    DEF = auto()
    RETURN = auto()
+ NOT = auto()
```

`components/lexica.py`

MODIFY Current target lines: 8–16

```
KEYWORDS = {
    "if": TokenType.IF,
    "else": TokenType.ELSE,
    "while": TokenType.WHILE,
    "def": TokenType.DEF,
    "return": TokenType.RETURN,
+ "not": TokenType.NOT,
    "true": TokenType.BOOL_LITERAL,
    "false": TokenType.BOOL_LITERAL,
}
```

`components/ast_nodes.py`

ADD Current target lines: 39–50

```
@dataclass
class UnaryOp(ASTNode):
    op: str
    operand: ASTNode
```

`components/parser.py`

MODIFY Current target lines: 201–225

```
+ if tok.token_type == TokenType.NOT:
+     op_tok = self._advance()
+     operand = self._factor()
+     return UnaryOp(op="not", operand=operand, line=op_tok.line, column=op_tok.column)
```

`components/type_checker.py`

MODIFY Current target lines: 131–152

```
if isinstance(node, UnaryOp) and node.op == "not":
    operand_type = self._infer_expr_type(node.operand, scope)
    if operand_type != LanguageType.BOOLEAN:
        raise TypeCheckError(self._error_at(node, "Operator 'not' requires Boolean operand"))
    return LanguageType.BOOLEAN
```

`components/interpreter.py`

MODIFY Current target lines: 129–144

```
if isinstance(node, UnaryOp) and node.op == "not":
    return not self._evaluate(node.operand, environment)
```

`components/parse_tree_printer.py`

MODIFY Current target lines: 197–215

```
+ if isinstance(node, UnaryOp) and node.op == "not":
+     return ParseTreeNode("factor", [ParseTreeNode("'not'"), self._factor_node(node.operand)])
```

`components/ast_printer.py`

MODIFY Current target lines: 99–111

```
+ def _visit_UnaryOp(self, node: UnaryOp, lines: list[str], level: int) -> None:
+     lines.append(f"{self._pad(level)}UnaryOp '{node.op}'")
+     self._visit(node.operand, lines, level + 1)
```

`tests/test_pipeline.py`

ADD Current target lines: insert before line 453

```
def test_unary_not_boolean(self) -> None:
    result = run_pipeline("print(not false);")
    self.assertEqual(result.outputs, ["true : Boolean"])

def test_unary_not_rejects_integer(self) -> None:
    with self.assertRaises(TypeError):
        run_pipeline("print(not 1);")
```

Case L: Add Block Comments `/*...*/`

Trap: line counting must still be correct.

1. Edit only `components/lexica.py` first.
2. In `_scan_token()`, detect `/` followed by `*`.
3. Consume until `*/`; if a newline is seen, increment `self.line` and reset `self.column` exactly as the string scanner already does around lines 112–124.
4. Raise `LexerError` if the block comment never closes.
5. Add lexer tests for both normal skipping and unterminated comment failure.

`components/lexica.py`

MODIFY *Current target lines: 83–99*

```
+ if char == "/" and self._peek() == "*":
+     self._advance()
+     while not self._is_at_end():
+         if self._peek() == "\n":
+             self.line += 1
+             self.column = 1
+             self._advance()
+         continue
+     if self._peek() == "*" and self._peek_next() == "/":
+         self._advance()
+         self._advance()
+         return None
+     self._advance()
+     raise LexerError(self._error_message("Unterminated block comment"))
```

`tests/test_pipeline.py`

ADD *Current target lines: insert before line 85*

```
def test_block_comment_is_skipped(self) -> None:
    result = run_pipeline("x = 1; /* skip */ print(x);")
    self.assertEqual(result.outputs, ["1 : Integer"])

def test_unterminated_block_comment_raises(self) -> None:
    with self.assertRaises(LexerError):
        Lexer("/* never closes").tokenize()
```

Case M: Add a New UI/Pipeline Output Only

Trap: do not touch lexer/parser/type checker/interpreter if the language itself is unchanged.

1. Start in `components/pipeline.py`.
2. Add the new formatted output to `PipelineResult` if needed.
3. Update `main.py` if the CLI stage output must print it.
4. Update `ui.py` if the Tkinter notebook needs a new tab.
5. Add a targeted pipeline or integration test only if the output is testable.

`components/pipeline.py`

MODIFY *Current target lines: 15–22 and 52–78*

```
@dataclass
class PipelineResult:
    tokens: list[Token]
    parse_tree_text: str
    ast_text: str
    checked_scope: SymbolTable
    outputs: list[str]
+ summary_text: str

def run_pipeline(source_code: str) -> PipelineResult:
    tokens = Lexer(source_code).tokenize()
    tree = Parser(tokens).parse()
    parse_tree_text = ParseTreePrinter().print(tree)
    ast_text = ASTPrinter().print(tree)
    checked_scope = TypeChecker().check(tree)
    outputs: list[str] = []
    Interpreter(output_callback=outputs.append).execute(tree)
    summary_text = f"{len(tokens)} token(s), {len(outputs)} output line(s)"
```

```

    return PipelineResult(
        tokens=tokens,
        parse_tree_text=parse_tree_text,
        ast_text=ast_text,
        checked_scope=checked_scope,
        outputs=outputs,
        summary_text=summary_text,
    )

def format_stage_output(result: PipelineResult) -> str:
    # patch excerpt: insert these lines into the existing sections list
    sections = [
        "=" * 60,
        " STAGE 1 -- TOKENS",
        "=" * 60,
        format_tokens(result.tokens),
+ "=" * 60,
+ " SUMMARY",
+ "=" * 60,
+ result.summary_text,
    ]

```

ui.py

MODIFY Current target lines: 94–99 and 141–146

```

self.output_text = self._make_output_tab(notebook, "Execution Output")
self.tokens_text = self._make_output_tab(notebook, "Tokens")
self.parse_tree_text = self._make_output_tab(notebook, "Parse Tree")
self.ast_text = self._make_output_tab(notebook, "AST")
self.types_text = self._make_output_tab(notebook, "Type Table")
self.summary_text = self._make_output_tab(notebook, "Summary")
self.error_text = self._make_output_tab(notebook, "Errors")

self._write_text(self.output_text, "\n".join(result.outputs) if result.outputs else "(no output)")
self._write_text(self.tokens_text, format_tokens(result.tokens))
self._write_text(self.parse_tree_text, result.parse_tree_text)
self._write_text(self.ast_text, result.ast_text)
self._write_text(self.types_text, result.checked_scope.format_table())
self._write_text(self.summary_text, result.summary_text)

```

main.py

MODIFY Current target lines: 54–59

```

# no parser/type checker change needed
# CLI already prints whatever format_stage_output includes

```

tests/test_pipeline.py

ADD Current target lines: insert before line 453

```

def test_pipeline_summary_output_present(self) -> None:
    result = run_pipeline("x = 1; print(x);")
    self.assertIn("token(s)", result.summary_text)

```

8. Where Specific Logic Lives

Question in exam	Start here first
“Why is -x working?”	components/parser.py, <code>_factor()</code> unary-minus desugaring.
“Where is type inference done?”	components/type_checker.py, especially <code>_assign_variable_type()</code> and <code>_infer_function_call_type()</code> .
“Why is 7/2 equal to 3?”	components/interpreter.py, <code>_evaluate_binary()</code> uses floor division.
“Why does <code>print(greet())</code> fail?”	components/type_checker.py, <code>_infer_builtin_print_type()</code> .
“Where is scope handled?”	components/symbol_table.py and interpreter class <code>_Environment</code> .
“Why are uncalled functions still checked?”	components/type_checker.py, <code>_check_uncalled_functions()</code> .
“Where do CLI and UI share code?”	components/pipeline.py.

9. High-Value Limitations to Mention in Answers

- No recursion support.
- No logical operators such as `and`, `or`, `not`.
- No relational operators such as `<`, `>`, `<=`, `>=`.
- No comments in the language.

- No explicit type annotations.
- Comparisons are limited to numeric same-type operands.
- Nested function definitions are parseable as statements but not supported by the type checker or interpreter as a feature design target; safest assumption is top-level function definitions only.

10. Minimal Test Strategy for Any New Feature

When you finish a code-test change, add at least these test shapes in `tests/test_pipeline.py`:

1. lexer tokenization test
2. parser or precedence test if syntax changed
3. type checker success test
4. type checker failure test
5. integration test that reaches execution
6. runtime error test if division, indexing, or similar unsafe behavior exists

11. Final Audit Checklist

Use this before trusting your exam answer:

1. Did you name the exact file for every change?
2. Did you state whether the file is **ADD**, **MODIFY**, or **NO CHANGE**?
3. Did you point to the current target lines in the existing code?
4. If grammar changed, did you update both parser and parse tree printer?
5. If semantics changed, did you update both type checker and interpreter, or explicitly state why one stays unchanged?
6. If a new statement was introduced, did you consider AST nodes, parser dispatch, type checker statement handling, and interpreter statement execution?
7. If a new operator was introduced, did you consider tokenization, precedence, type rule, runtime rule, and tests?
8. If a built-in was introduced, did you avoid unnecessary parser changes?
9. Did you add at least one success test and one failure test?
10. Did you check for hidden traps like precedence, scope, void values, or line counting?

12. Improvements Over Version 1

- Added likely code-test questions.
- Added harder trap cases.
- Added per-file snippets for the highest-probability extensions.
- Added exact current line references.
- Marked each snippet as **ADD**, **MODIFY**, or **NO CHANGE**.
- Added before/after comparison blocks for key edits.
- Strengthened changed-line highlighting beyond plain + and ~.
- Kept the answer grounded in this repository, not generic compiler notes.

13. One-Page Memory Aid

- **New syntax token?** Edit `tokens.py`, then `lexica.py`.
- **New grammar?** Edit `parser.py`; maybe also `ast_nodes.py`.
- **New tree output broken?** Update both printers.
- **New semantic rule?** Edit `type_checker.py`.
- **New runtime behavior?** Edit `interpreter.py`.

- **Need proof?** Add tests in `tests/test_pipeline.py`.
- **UI or CLI only?** Most likely `pipeline.py`, `main.py`, or `ui.py`.

Best exam habit: before changing code, identify whether the requested feature is **lexical**, **syntactic**, **semantic**, **runtime**, or a mix. Then modify files in pipeline order and finish by adding tests.